

Compute konfigurieren

Diese Seite ist ein Referenzleitfaden für alle Einstellungsmöglichkeiten beim Erstellen von All-Purpose- und Jobs-Compute (Classic Compute) über die Benutzeroberfläche. Wer eigene Cluster konfiguriert, findet hier die wichtigsten Stellschrauben im Überblick.

Compute-Policy

Beim Erstellen von Compute wählt man zunächst eine Policy (Richtlinie) aus, die festlegt, welche Konfigurationsoptionen überhaupt verfügbar sind. Standardmäßig ist die Policy "Personal Compute" ausgewählt, die einfache Einzelmaschinen-Ressourcen ermöglicht. Workspace-Administratoren verwalten den Zugriff auf weitere Policies.

Performance-Einstellungen

Databricks-Runtime-Version

Die Runtime-Version bestimmt unter anderem die enthaltene Apache-Spark-Version. Empfehlungen:

- Für All-Purpose-Compute: aktuelle Versionen verwenden, um von den neuesten Optimierungen zu profitieren.
- Für produktive Jobs: LTS-Versionen (Long Term Support) verwenden, da diese länger stabil unterstützt werden.
- Für Data-Science- und Machine-Learning-Workloads: spezielle Databricks-Runtime-ML-Versionen verwenden.

Photon-Beschleunigung

Photon ist standardmäßig aktiviert auf Compute-Ressourcen mit Databricks Runtime 9.1 LTS oder neuer. Über eine Checkbox lässt sich die Funktion an- oder abschalten. Mehr dazu im eigenen Kapitel zu Photon.

Worker-Knotentypen

Eine Compute-Ressource besteht aus einem Driver-Knoten (Steuerungsknoten) und null oder mehr Worker-Knoten. Instanztypen können für Driver und Worker getrennt ausgewählt werden.

- **Flexible Node Types:** Fallen automatisch auf alternative, kompatible Instanztypen zurück, wenn der gewünschte Typ gerade nicht verfügbar ist – das erhöht die Zuverlässigkeit beim Starten.
- **GPU-Instanzen:** Für rechenintensive Aufgaben (z. B. Machine Learning). P2-Instanzen werden nicht mehr unterstützt.
- **AWS-Graviton-Instanzen:** ARM64-basierte Prozessoren mit gutem Preis-Leistungs-Verhältnis. Unterstützt ab Databricks Runtime 9.1 LTS (für Photon ab 10.2 LTS).
- **Fleet Instance Types:** Variable Instanztypen, die über die AWS Spot Placement Score API automatisch die beste verfügbare, größenäquivalente Option ermitteln.

Single-Node-Compute

Über eine Checkbox lässt sich eine Einzelknoten-Ressource erstellen mit folgenden Eigenschaften:

- Lokale Spark-Ausführung
- Der Driver fungiert gleichzeitig als Master und als Worker
- Ein Executor-Thread pro logischem Kern (abzüglich eines Kerns für den Driver)
- Alle Logs werden in den Driver-Logs gespeichert
- Eine Umwandlung zu Multi-Node ist nicht möglich

Geeignet für kleine Datenmengen und nicht-verteilte Workloads, nicht geeignet für große, verteilte Verarbeitung.

Autoscaling aktivieren

Autoscaling passt die Anzahl der Worker-Knoten dynamisch zwischen einem festgelegten Minimum und Maximum an, je nach aktuellem Bedarf.

Vorteile:

- Schnellere Fertigstellung von Workloads im Vergleich zu unterdimensionierten, statischen Ressourcen
- Geringere Kosten im Vergleich zu statisch überdimensionierten Ressourcen
- Hohe Auslastung ohne manuelles Nachjustieren

Verhalten:

- Premium-Plan-Workspaces nutzen optimiertes Autoscaling: Hochskalieren in maximal 2 Schritten, Herunterskalieren basierend auf dem Anteil ungenutzter Knoten.
- Standard-Plan-Workspaces nutzen klassisches Autoscaling mit exponentiellem Skalierungsmuster.
- Jobs-Compute skaliert nach 40 Sekunden Unterauslastung herunter, All-Purpose-Compute nach 150 Sekunden.

- Autoscaling kann nicht für `spark-submit` -Jobs aktiviert werden.

Wichtig: Apache Spark Dynamic Allocation sollte nicht zusätzlich aktiviert werden – Databricks-Autoscaling übernimmt bereits die Verwaltung der Worker-Knoten.

Erweiterte Performance-Einstellungen

- **Spot-Instanzen:** Über eine Checkbox aktivierbar. Die erste Instanz ist immer On-Demand (der Driver-Knoten läuft grundsätzlich immer On-Demand, nie als Spot-Instanz).
- **Automatische Terminierung:** Die Compute-Ressource beendet sich automatisch, wenn die festgelegte Anzahl an Inaktivitäts-Minuten seit dem letzten ausgeführten Befehl überschritten wird.
- **Driver-Typ:** Eigene Instanztyp-Auswahl für den Driver-Knoten. Standardmäßig identisch mit dem Worker-Typ, kann aber hochskaliert werden, wenn im Notebook große Datenmengen gesammelt und analysiert werden.

Tags

Benutzerdefinierte Schlüssel-Wert-Paare zur Kostenüberwachung und -zuordnung nach Organisationseinheiten. Bei Compute, die über einen Instance Pool gestartet wird, gelten Tags nur für die DBU-Abrechnungsberichte, nicht für die zugrunde liegenden Cloud-Ressourcen.

Erweiterte Einstellungen

Zugriffsmodi

Modus	Beschreibung
Standard	Mehrbenutzerzugriff mit Datenisolation; unterstützt Python, SQL, Scala
Dedicated	Zugriff nur für einen Nutzer/eine Gruppe; unterstützt Python, SQL, Scala, R

Die Einstellung "Auto" wählt standardmäßig "Standard", außer es wird eine ML-Runtime, eine GPU-Instanz oder eine Runtime älter als 14.3 ausgewählt – dann wird automatisch "Dedicated" verwendet.

Instanzprofile

IAM-Rollen, die einen sicheren Zugriff auf AWS-Ressourcen ermöglichen, ohne Zugriffsschlüssel im Code hinterlegen zu müssen. Nach der Konfiguration wird das Profil aus einem Dropdown-Menü

ausgewählt. Überprüfen lässt sich der Zugriff z. B. mit dem Befehl `dbutils.fs.ls("s3a://<bucket-name>/")`.

Verfügbarkeitszonen

Die Einstellung "Auto" wählt automatisch eine Zone basierend auf verfügbaren Subnetz-IP-Adressen im Workspace aus und versucht bei unzureichender Kapazität automatisch andere Zonen. Eine feste Zonenauswahl ist sinnvoll, um reservierte Instanzen optimal zu nutzen.

AWS Capacity Blocks

Reservierte Rechenkapazität lässt sich über spezielle Tags (Schlüssel: `X-Databricks-AwsCapacityBlockId`) sowie passende Verfügbarkeitszonen konfigurieren.

Autoscaling Local Storage

Databricks hängt automatisch zusätzliche EBS-Volumes an, wenn Worker-Knoten wenig Festplattenspeicher haben – bis zu insgesamt 5 TB pro Instanz. Die Volumes werden erst bei Terminierung der Instanz wieder getrennt.

EBS-Volumes

Standardkonfiguration:

- 30 GB verschlüsseltes Root-Volume für die Instanz
- 150 GB verschlüsseltes Root-Volume für Container
- Optional 75 GB Worker-Log-Volume (nur für HIPAA-konforme Umgebungen)

Zusätzlich lassen sich General-Purpose-SSD-Shuffle-Volumes für Jobs mit großen Ausgabedaten hinzufügen. Optional können auch kundenverwaltete Verschlüsselungsschlüssel genutzt werden.

Verschlüsselung lokaler Datenträger

Eine Public-Preview-Funktion, die lokal angehängten Speicher verschlüsselt. Aktivierbar über `enable_local_disk_encryption: true` in der Clusters-API. Durch die Ver-/Entschlüsselung ist mit einem gewissen Performance-Verlust zu rechnen.

Spark-Konfiguration

Benutzerdefinierte Spark-Eigenschaften werden als Schlüssel-Wert-Paare (eine Zeile pro Eigenschaft) konfiguriert. Im Standard-Access-Mode sind ab Databricks Runtime 19 bestimmte Spark-

Konfigurationseigenschaften eingeschränkt. Für das Abrufen von Secrets gibt es die Syntax `spark.<property> {{secrets/<scope>/<name>}}`.

Compute-Log-Auslieferung

Logs werden alle 5 Minuten übertragen und stündlich archiviert. Mögliche Ziele:

- **Volumes** (empfohlen): Unity-Catalog-Pfade
- **S3**: Amazon-S3-Buckets (benötigt passende Instanzprofil-Berechtigungen)
- **DBFS**: Legacy-Pfade im Databricks File System

Die Logs werden in einem Unterordner abgelegt, der nach der Cluster-ID benannt ist.

Umgebungsvariablen

Benutzerdefinierte Variablen, die z. B. von Init-Skripten genutzt werden können. Im Standard-Access-Mode ist ab Databricks Runtime 19 nur eine vordefinierte Auswahl an Umgebungsvariablen verfügbar.

Wichtige Einschränkungen im Überblick

- Spot-Instanzen sollten nicht für den Driver-Knoten verwendet werden.
- Apache Spark Dynamic Allocation sollte nicht gemeinsam mit Databricks-Autoscaling aktiviert werden.
- GPU-Scheduling steht bei Single-Node-Compute nicht zur Verfügung.
- Bei Parquet-Dateien mit benutzerdefinierten Typen (UDT-Spalten) muss auf Single-Node-Ressourcen der native Reader deaktiviert werden.

Quelle: <https://docs.databricks.com/aws/en/compute/configure> · Stand: 2026-08-06